Digital Computer Logic Final Project

MINI CPU

Presented By จอมมารโบ่

- 6933076221 Natthatida Hutangkoon
- 6933277021 Sukrit Tongsri
- 6933212221 Phuripong Deeted
- 6933064721 Chanawat Chobtumkit



โจทย์

CEDT Final Project 2569 Digital Logic 2110252: Mini CPU + Stack

ภาพรวมโจทย์

ให้นิสิตสร้าง CPU ขนาด 8 bits ด้วยโปรแกรม Digital ขั้นแรกวงจรรอรับโปรแกรมมาเก็บใน RAM ขนาด 256 x 14 bits (pRAM – Program RAM) แล้วทำตามคำสั่งตั้งแต่ address 0x00 ไปจนเจอคำสั่ง STOP จากนั้นรอสัญญาณให้ส่งคำตอบ แล้วแสดงผลลัพธ์ที่เก็บใน RAM ขนาด 256 x 8 bits (rRAM – Result RAM) ตำแหน่ง 0x00–0x0F ออกทาง output และ 7-segment 2 ตัว
CPU นี้มีองค์ประกอบหลักดังนี้

- register 8 bits: accA, accB (accumulator), regC, regD และ SP (Stack Pointer)
- flag 3 ตัว: eq, gr, ls ใช้กับคำสั่ง jump
- stack อยู่ใน rRAM ช่วงบน ใช้ผ่านคำสั่ง PUSH / POP
- คำสั่งพื้นฐาน 42 คำสั่ง จัดกลุ่มตาม bit ของ opcode
- ช่วงท้ายโครงงานจะมีคำสั่งพิเศษ (Secret) ปล่อยเพิ่มในช่อง opcode ที่สงวนไว้ ควรออกแบบวงจรให้ต่อยอดได้

accumulator คือ register ที่เก็บค่าส่งเข้า ALU และรับผลลัพธ์กลับมา (อ่านเพิ่ม: https://en.wikipedia.org/wiki/Accumulator_(computing))

รูปแบบคำสั่งและ Opcode

คำสั่งขนาด 14 bits แบ่งเป็น opcode 6 bits [13:8] และ operand 8 bits [7:0] มีคำสั่งพื้นฐาน 42 คำสั่ง จัดกลุ่มตาม bit ของ opcode ดังนี้

- opcode[5:4] = กลุ่มคำสั่ง (00 Data, 01 Memory/Stack, 10 ALU, 11 Control/Special)
- opcode[3:2] = ชุดคำสั่งย่อยภายในกลุ่ม ชุดละ 4 ช่อง
- opcode[1:0] = คำสั่งในชุด ช่องที่เขียนว่า “ยังไม่ใช้” หรือ “สงวนไว้” จะไม่ปรากฏใน testcase ของคำสั่งพื้นฐาน

กลุ่ม [5:4]	ชุด [3:2]	ชื่อชุด	จำนวนคำสั่ง
00 Data	00	Immediate	3
00 Data	01	คัดลอกจาก accA / accB	4
00 Data	10	คัดลอกเข้า accA	3
00 Data	11	อ่าน Input	2
01 Memory/Stack	00	Direct memory	3
01 Memory/Stack	01	Indirect memory	2
01 Memory/Stack	10	Stack	2
01 Memory/Stack	11	(สงวนไว้)	0
10 ALU	00	Arithmetic	3
10 ALU	01	Division	2
10 ALU	10	Logic	4
10 ALU	11	Shift / Rotate	4
11 Control	00	Branch	4
11 Control	01	Loop	1
11 Control	10	Compare / Test	2
11 Control	11	คำสั่งพิเศษ + STOP	3

กลุ่ม 00 : Data (12 คำสั่ง) — opcode[5:4] = 00

00	**Immediate (3)**	โหลดค่าคงที่ 8 บิตเข้า accA, accB, regC
01	**คัดลอกจาก acc (4)**	ย้ายข้อมูลจาก accA/accB ไประเบียบอื่น
10	**คัดลอกเข้า acc (3)**	ย้ายข้อมูลจากรีจิสเตอร์ต่างๆ เข้าสู่ accA
11	**อ่าน Input (2)**	รับค่าจากพอร์ตอินพุตภายนอก M และ N

กลุ่ม 01 : Memory / Stack / Subroutine — opcode[5:4] = 01

00	**Direct memory (3)**	อ่าน/เขียน rRAM แบบระบุแอดเดรสตรง
01	**Indirect memory (2)**	อ้างอิงแอดเดรสผ่าน Pointer (regC, regD)
10	**Stack (2)**	คำสั่ง PUSH, POP ปรับค่า SP อัตโนมัติ
11	**Subroutine (2)**	CALL (เก็บ PC ลง Stack) และ RET (ดึง PC กลับ)

กลุ่ม 10 : ALU (13 คำสั่ง) — opcode[5:4] = 10

00	**Arithmetic (3)**	บวก (ADD), ลบ (SUB), คูณ (MUL)
01	**Division (2)**	หารเอาผลลัพธ์ (DIV) และหารเอาเศษ (MOD)
10	**Logic (4)**	บิตไวส์ AND, OR, XOR, NOT
11	**Shift / Rotate (4)**	เลื่อนและหมุนบิตซ้าย/ขวา (SHL, SHR, ROL, ROR)

กลุ่ม 11 : Control / Special (ครบ 4 ช่องเต็ม) — opcode[5:4] = 11

00	**Branch (4)**	JMP, JEQ, JGR, JLS ตรวจสอบแฟล็ก
01	**Loop (1)**	DJNZ (ลดค่า regD และ Jump ถ้ายังไม่ศูนย์)
10	**Compare / Test (2)**	CMP (เทียบ A กับ B), isPrime (เช็คจำนวนเฉพาะ)
11	**Special + STOP (4)**	**GCD, FIB, SSUM (Secret), STOP**

รูปแบบคำสั่ง 14 บิต (Instruction Bitfield)

บิต 13 – 8 (6 บิต)	บิต 7 – 0 (8 บิต)
Opcode [13:8] (รหัสสั่งการ)	**Operand [7:0] (ค่าคงที่ / แอดเดรส)**

opcode[5:4] (2 บิต)

กลุ่มคำสั่งหลัก (4 กลุ่ม)

- **00:** Data (ย้าย/โหลดข้อมูล)
- **01**: Memory / Stack
- **10**: ALU (คำนวณคณิต/ตรรกะ)
- **11**: Control / Special

opcode[3:2] (2 บิต)

ชุดคำสั่งย่อยภายในกลุ่ม

- แต่ละกลุ่มแบ่งเป็น 4 ชุดย่อย
- เช่น ในกลุ่ม 10 (ALU):
  00: Arithmetic, 01: Division
  10: Logic, 11: Shift/Rotate

opcode[1:0] (2 บิต)

คำสั่งในชุด / ช่องสงวน

- กำหนดฟังก์ชันย่อยในชุด (สูงสุด 4 คำสั่ง)
- ช่องที่เขียนว่า **"ยังไม่ใช้"** หรือ **"สงวนไว้"** จะถูกใช้ปล่อยโจทย์พิเศษ (Secret Instructions) ในช่วงท้าย

แนวคิดการออกแบบ CPU

ขั้นที่ 1

สเปก & รูปแบบคำสั่ง

วางข้อกำหนดพื้นฐานทางสถาปัตยกรรม

- **กำหนดขนาดบัส:** Data 8-bit และ Inst 14-bit
- **แจกแจง Opcode:** 42 คำสั่งแบ่ง 4 กลุ่มหลัก
- **Memory Layout:** แบ่งโซน pRAM และ rRAM

เป้าหมาย: ได้ผัง Instruction Set ครบถ้วน

ขั้นที่ 2

สร้างบล็อก Datapath

สร้างส่วนคำนวณและเก็บข้อมูล

- **สร้างโมดูล ALU:** Add, Sub, Mul, Div, Logic
- **ต่อ Registers:** accA/B, regC/D, SP, PC
- **วางบัสข้อมูล:** MUX คัดเลือกแหล่งข้อมูลเข้า

เป้าหมาย: วงจรคำนวณทำงานถูกต้องเดี่ยวๆ

ขั้นที่ 3

สร้าง Control Unit

สร้างสมองสั่งการและจังหวะเวลา

- **ออกแบบ FSM:** สร้างสเตท 000 ถึง 111
- **ใส่ Decoder ROM:** แปลง Opcode เป็นสัญญาณคุม
- **กระจายสัญญาณ:** เดินสาย Clock Enable และ MUX

เป้าหมาย: รันคำสั่งพื้นฐานและกระโดดได้

ขั้นที่ 4

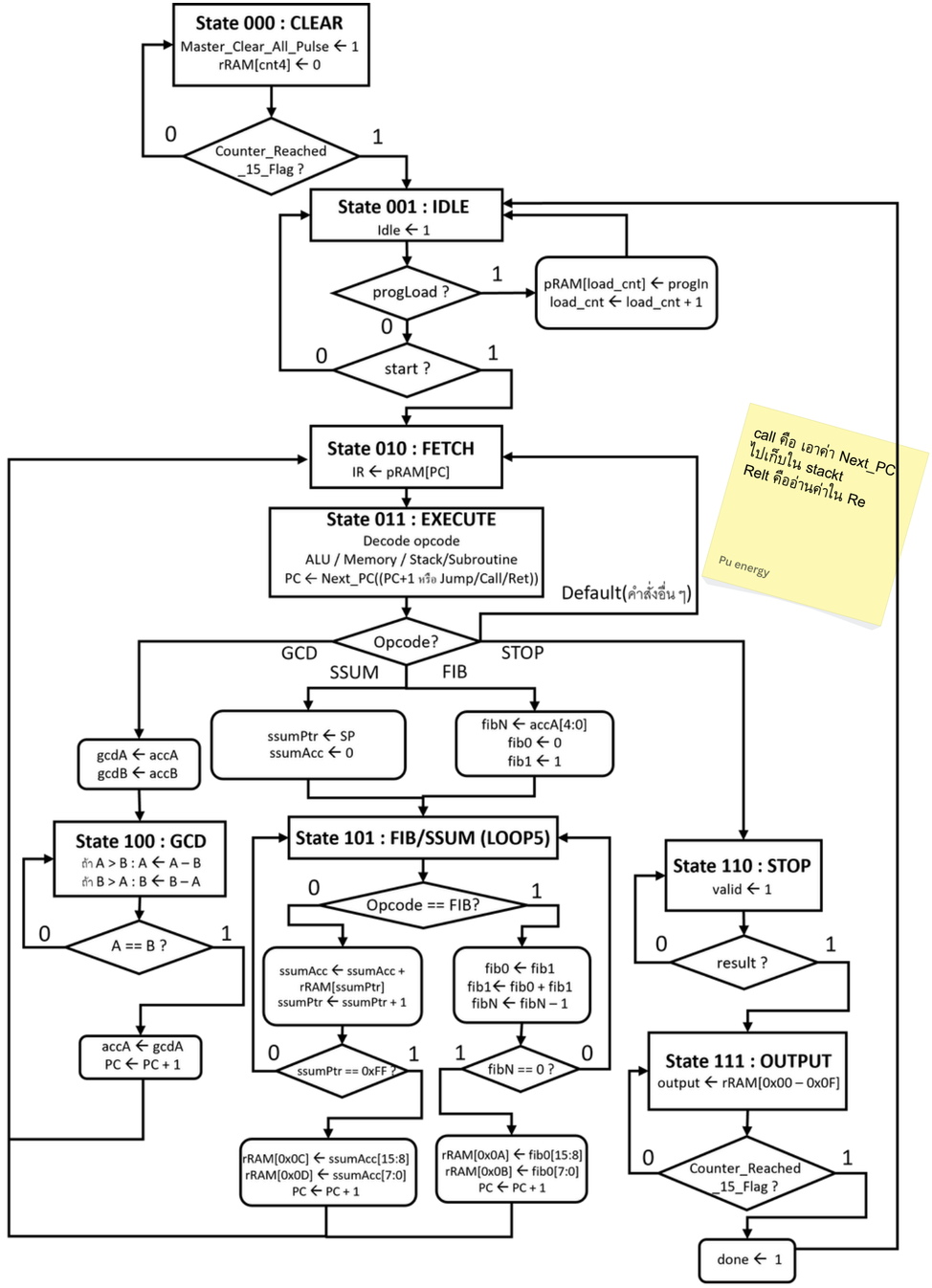
ฮาร์ดแวร์ลูป & ทดสอบรวม

ต่อยอดฟังก์ชันพิเศษและเชื่อมต่อระบบ

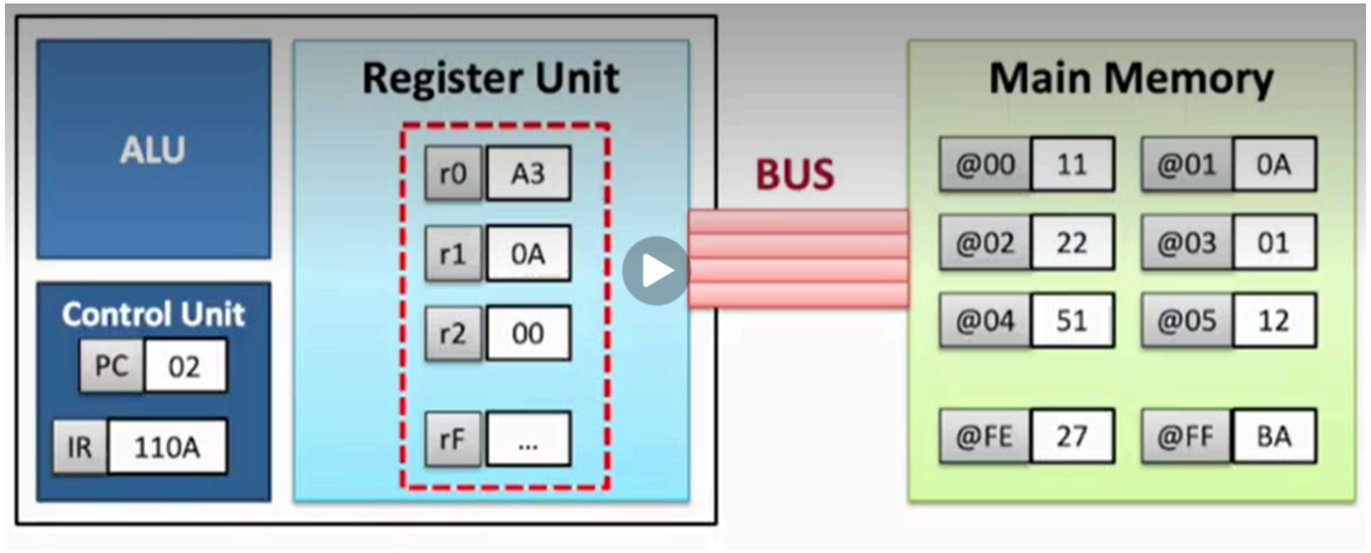
- **ต่อ Dedicated Loop:** วงจรเร่ง GCD, FIB, isPrime
- **ระบบแสดงผล:** สตรีม rRAM ออก 7-Segment
- **System Verification:** รันทดสอบโปรแกรมครบถ้วน

เป้าหมาย: ผ่าน Testcases พร้อมคำสั่ง Secret

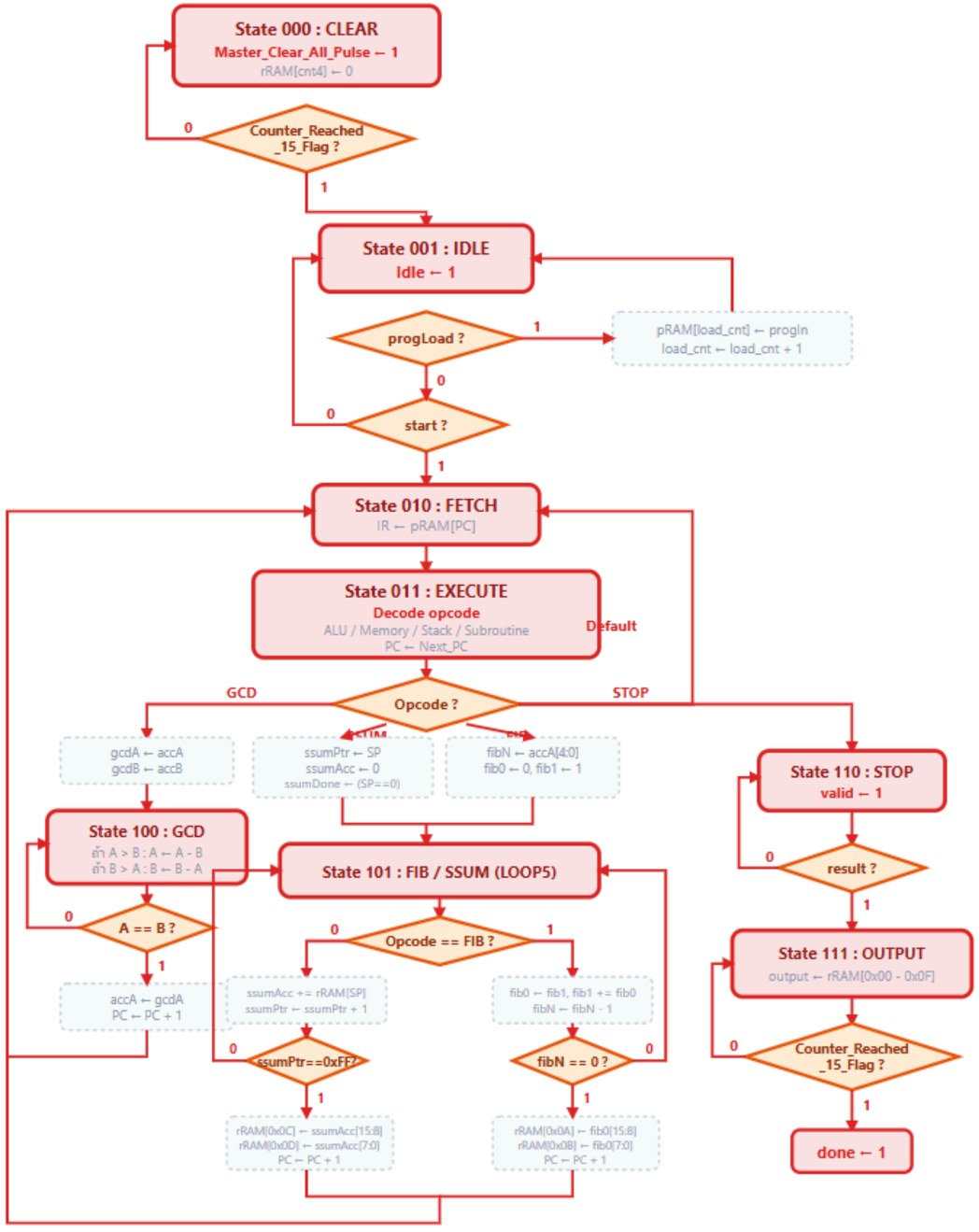
ASM Chart

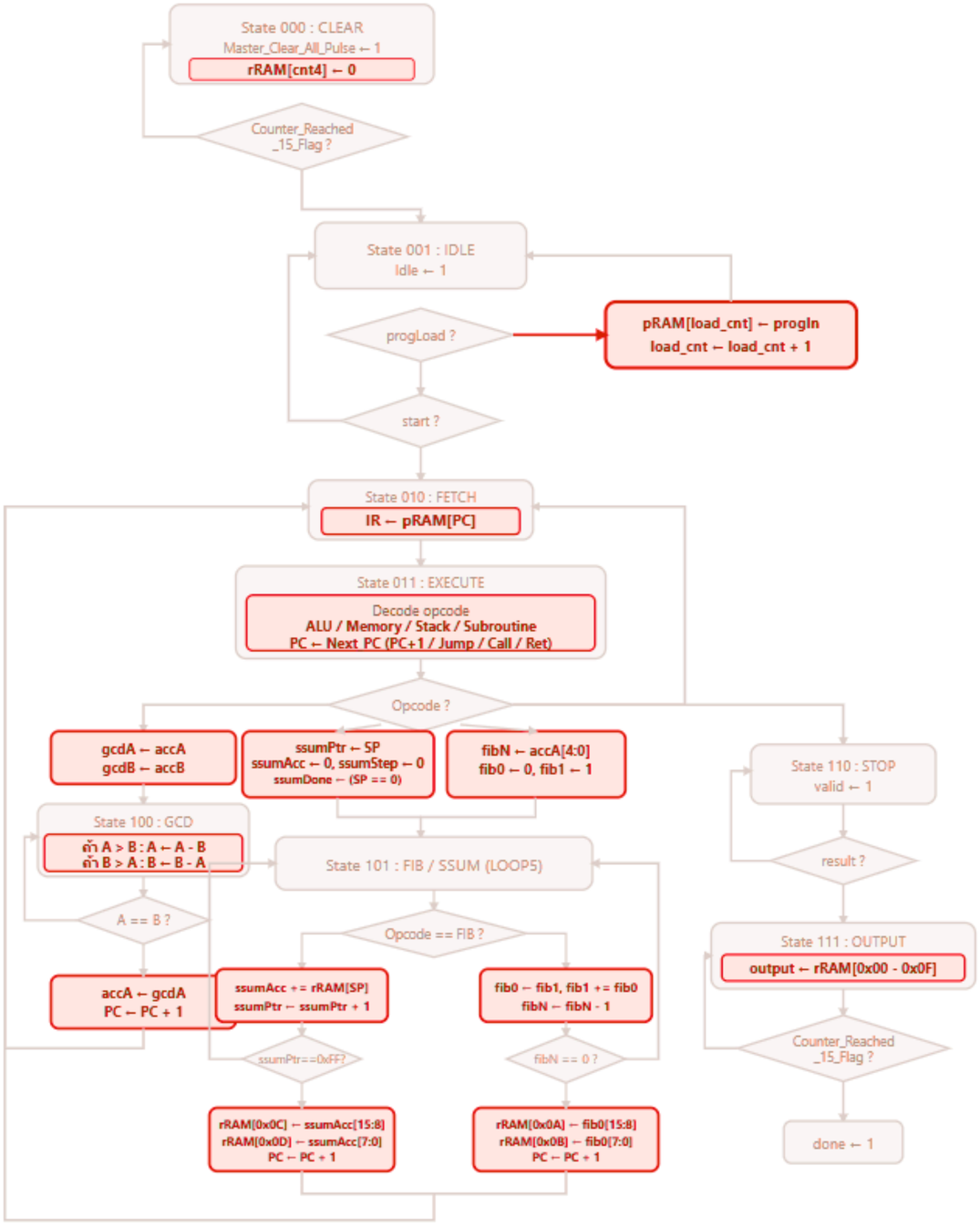


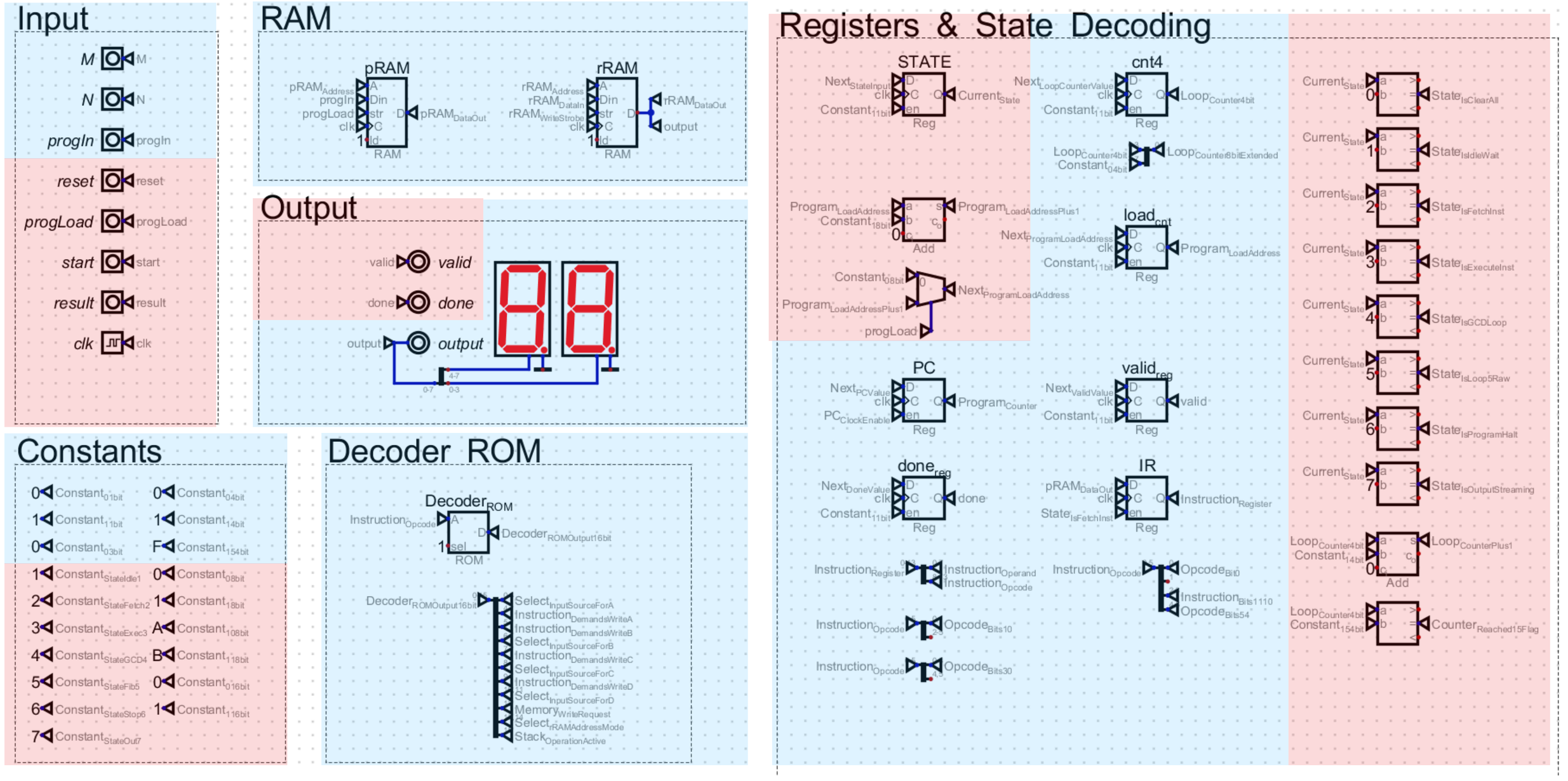
สถานะ (State)	หน้าที่ส่วน Datapath	หน้าที่ส่วน Control Unit
000 : CLEAR	เคลียร์ค่า rRAM[cnt4] และบอร์ดรีจิสเตอร์	ส่งพัลส์ Master Clear, เช็ค Counter 15
001 : IDLE	เขียน pRAM จาก progIn, เพิ่มค่า load_cnt	ขับสัญญาณ Idle=1, ตรวจจับ progLoad และ start
010 : FETCH	อ่าน pRAM[PC] แล้วโหลดค่าเข้าสู่ IR	สั่ง Enable IR และส่งแอดเดรส PC สู่ pRAM
011 : EXECUTE	ALU คำนวณ, เข้าถึง Memory/Stack, คำนวณ Next PC	ถอดรหัส Opcode และแตกกิ่งไป GCD, FIB, STOP
100 : GCD	ลบค่า A–B หรือ B–A, โอนผลลัพธ์เข้า accA	เช็คเงื่อนไข A == B เพื่อวนลูปหรือออกจากลูป
101 : FIB	คำนวณ F0+F1, เขียนไบต์ลง rRAM[0x0A, 0x0B]	สลับไบต์เขียนและเช็ค Fibonacci Finished Flag
110 : STOP	คงค่าของระบบ รอคำขอส่งผลลัพธ์	ขับสัญญาณ valid=1, ตรวจจับสัญญาณ result=1
111 : OUTPUT	อ่าน rRAM[0x00–0x0F] ขับออก Output Bus	วนลูป 16 รอบ และขับ done=1 เมื่อเสร็จสิ้น



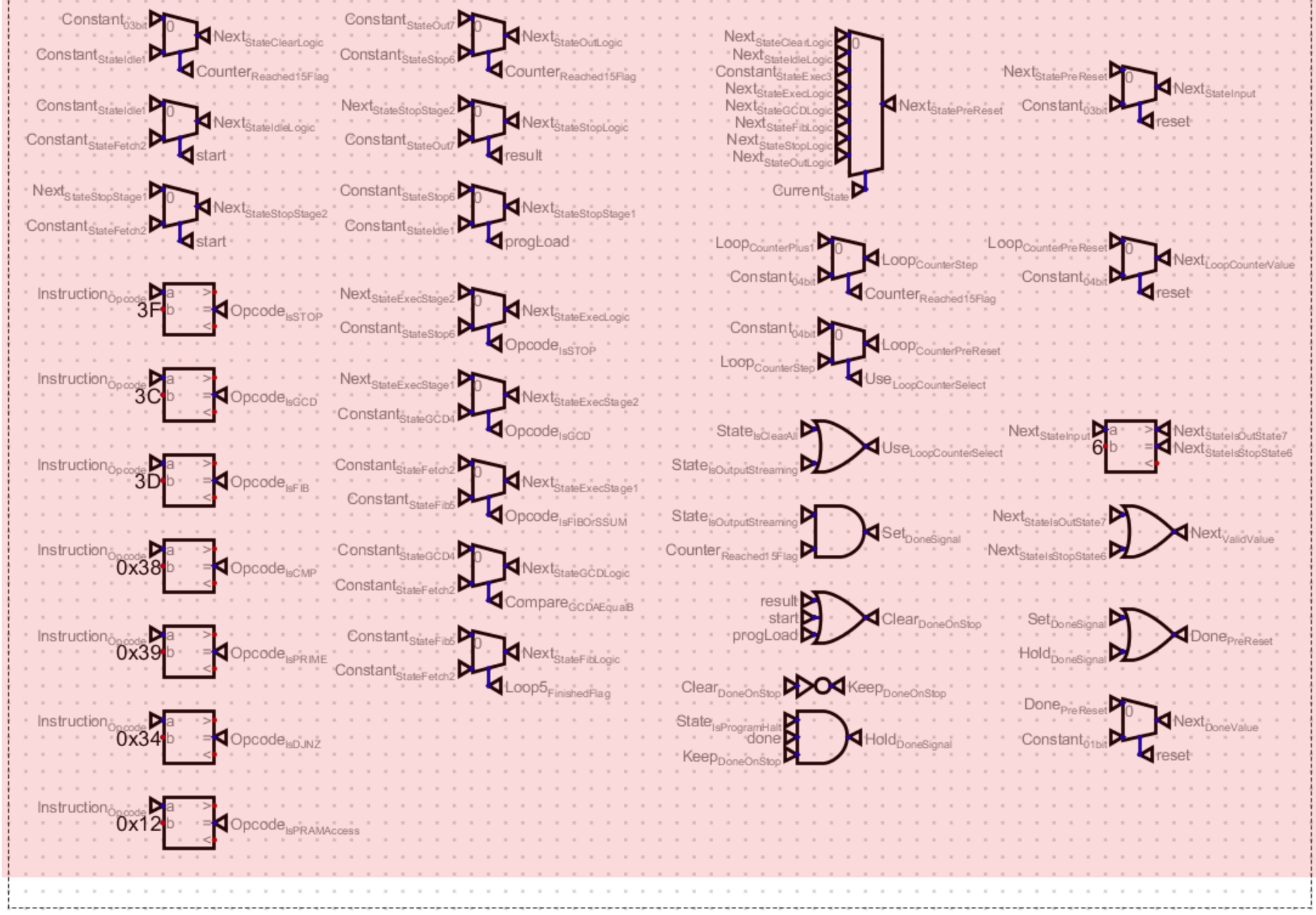
Datapath & Control Unit


State 000 : CLEAR
Master_Clear_All_Pulse ← 1
rRAM[cnt4] ← 0
Counter_Reached _15_Flag ?
0
1
State 001 : IDLE
Idle ← 1
progLoad ?
1
pRAM[load_cnt] ← progIn
load_cnt ← load_cnt + 1
0
start ?
0
1
State 010 : FETCH
IR ← pRAM[PC]
State 011 : EXECUTE
Decode opcode
ALU / Memory / Stack / Subroutine
PC ← Next_PC
Default
GCD
Opcode ?
STOP
gcdA ← accA
gcdB ← accB
ssumPtr ← SP
ssumAcc ← 0
ssumDone ← (SP==0)
fibN ← accA[4:0]
fib0 ← 0, fib1 ← 1
State 100 : GCD
ถ้า A > B : A ← A - B
ถ้า B > A : B ← B - A
State 101 : FIB / SSUM (LOOP5)
State 110 : STOP
valid ← 1
result ?
0
1
A == B ?
0
1
Opcode == FIB ?
0
1
State 111 : OUTPUT
output ← rRAM[0x00 - 0x0F]
accA ← gcdA
PC ← PC + 1
ssumAcc += rRAM[SP]
ssumPtr ← ssumPtr + 1
fib0 ← fib1, fib1 += fib0
fibN ← fibN - 1
ssumPtr==0xFF?
fibN == 0 ?
Counter_Reached _15_Flag ?
0
1
rRAM[0x0C] ← ssumAcc[15:8]
rRAM[0x0D] ← ssumAcc[7:0]
PC ← PC + 1
rRAM[0x0A] ← fib0[15:8]
rRAM[0x0B] ← fib0[7:0]
PC ← PC + 1
done ← 1

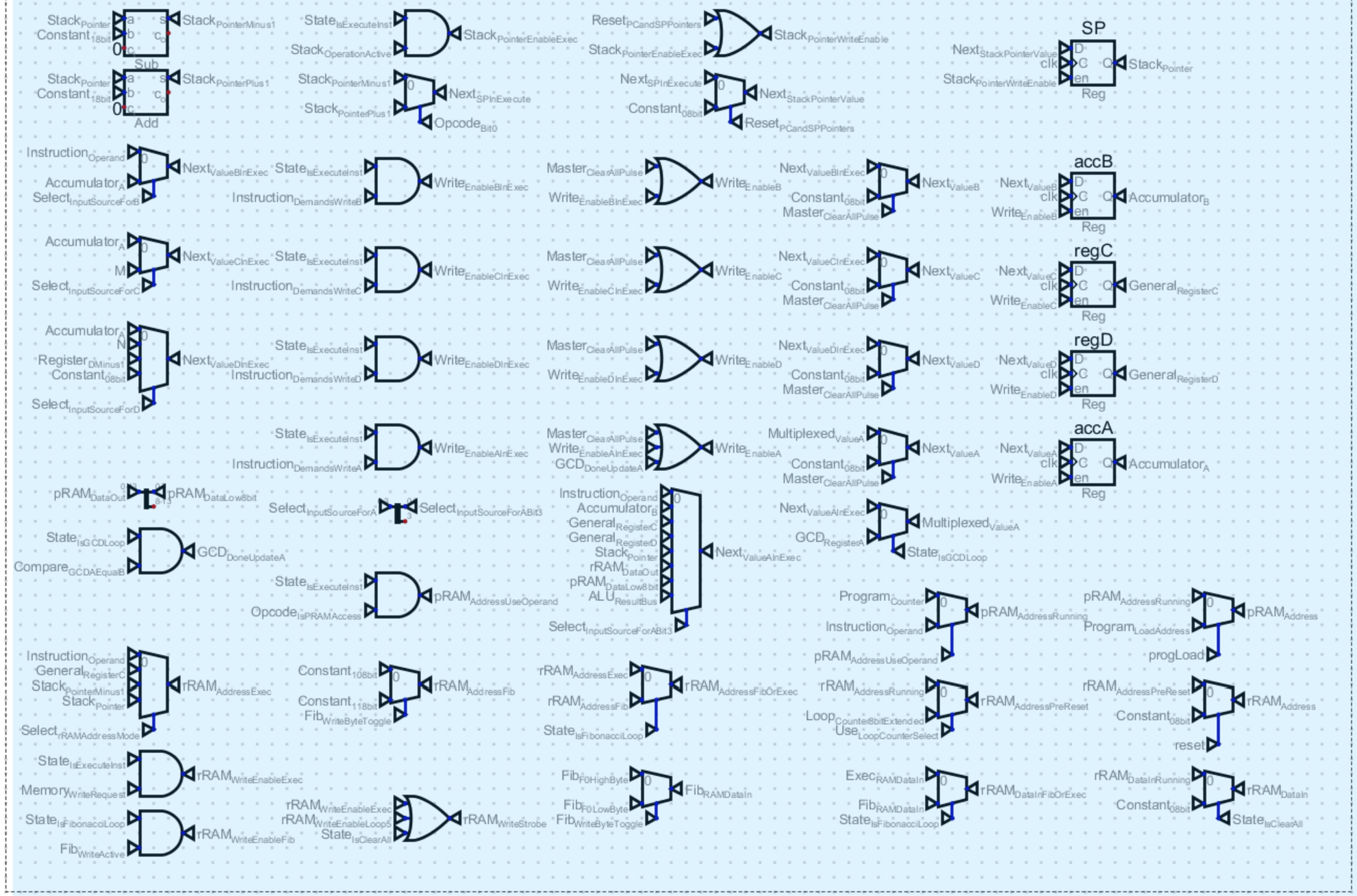

State 000 : CLEAR
Master_Clear_All_Pulse ← 1
rRAM[cnt4] ← 0
Counter_Reached _15_Flag ?
State 001 : IDLE
Idle ← 1
progLoad ?
pRAM[load_cnt] ← progIn
load_cnt ← load_cnt + 1
start ?
State 010 : FETCH
IR ← pRAM[PC]
State 011 : EXECUTE
Decode opcode
ALU / Memory / Stack / Subroutine
PC ← Next_PC (PC+1 / Jump / Call / Ret)
Opcode ?
gcdA ← accA
gcdB ← accB
ssumPtr ← SP
ssumAcc ← 0, ssumStep ← 0
ssumDone ← (SP == 0)
fibN ← accA[4:0]
fib0 ← 0, fib1 ← 1
State 110 : STOP
valid ← 1
State 100 : GCD
ถ้า A > B : A ← A - B
ถ้า B > A : B ← B - A
State 101 : FIB / SSUM (LOOP5)
result ?
A == B ?
Opcode == FIB ?
State 111 : OUTPUT
output ← rRAM[0x00 - 0x0F]
accA ← gcdA
PC ← PC + 1
ssumAcc += rRAM[SP]
ssumPtr ← ssumPtr + 1
fib0 ← fib1, fib1 += fib0
fibN ← fibN - 1
Counter_Reached _15_Flag ?
ssumPtr==0xFF?
fibN == 0 ?
rRAM[0x0C] ← ssumAcc[15:8]
rRAM[0x0D] ← ssumAcc[7:0]
PC ← PC + 1
rRAM[0x0A] ← fib0[15:8]
rRAM[0x0B] ← fib0[7:0]
PC ← PC + 1
done ← 1


Input
M
N
progIn
reset
progLoad
start
result
clk
RAM
pRAM
rRAM
Output
valid
done
output
Registers & State Decoding
STATE
cnt4
load_cnt
PC
valid_reg
done_reg
IR
Add
Reg
Constants
Decoder ROM
ROM

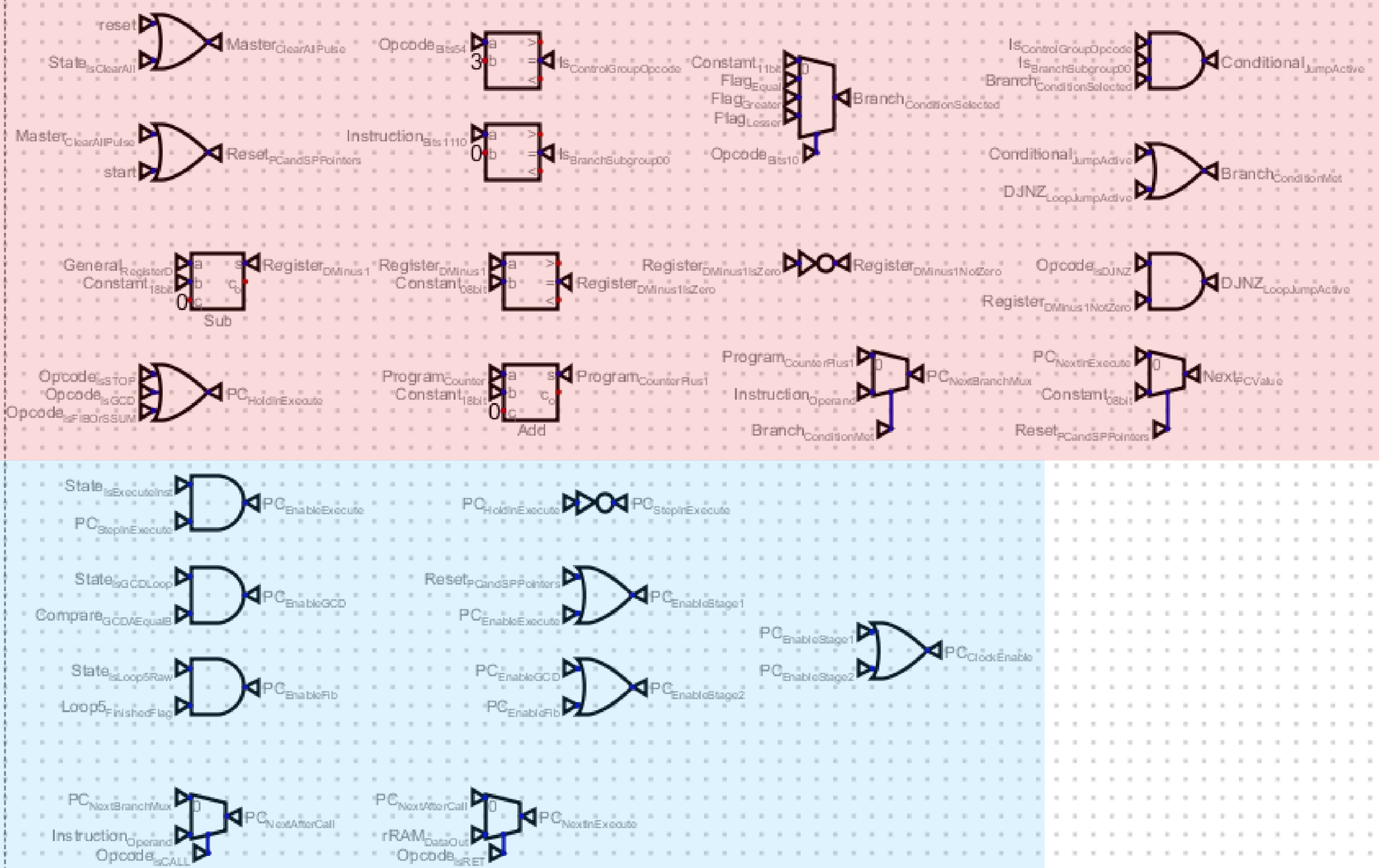
Next-State Logic & Control

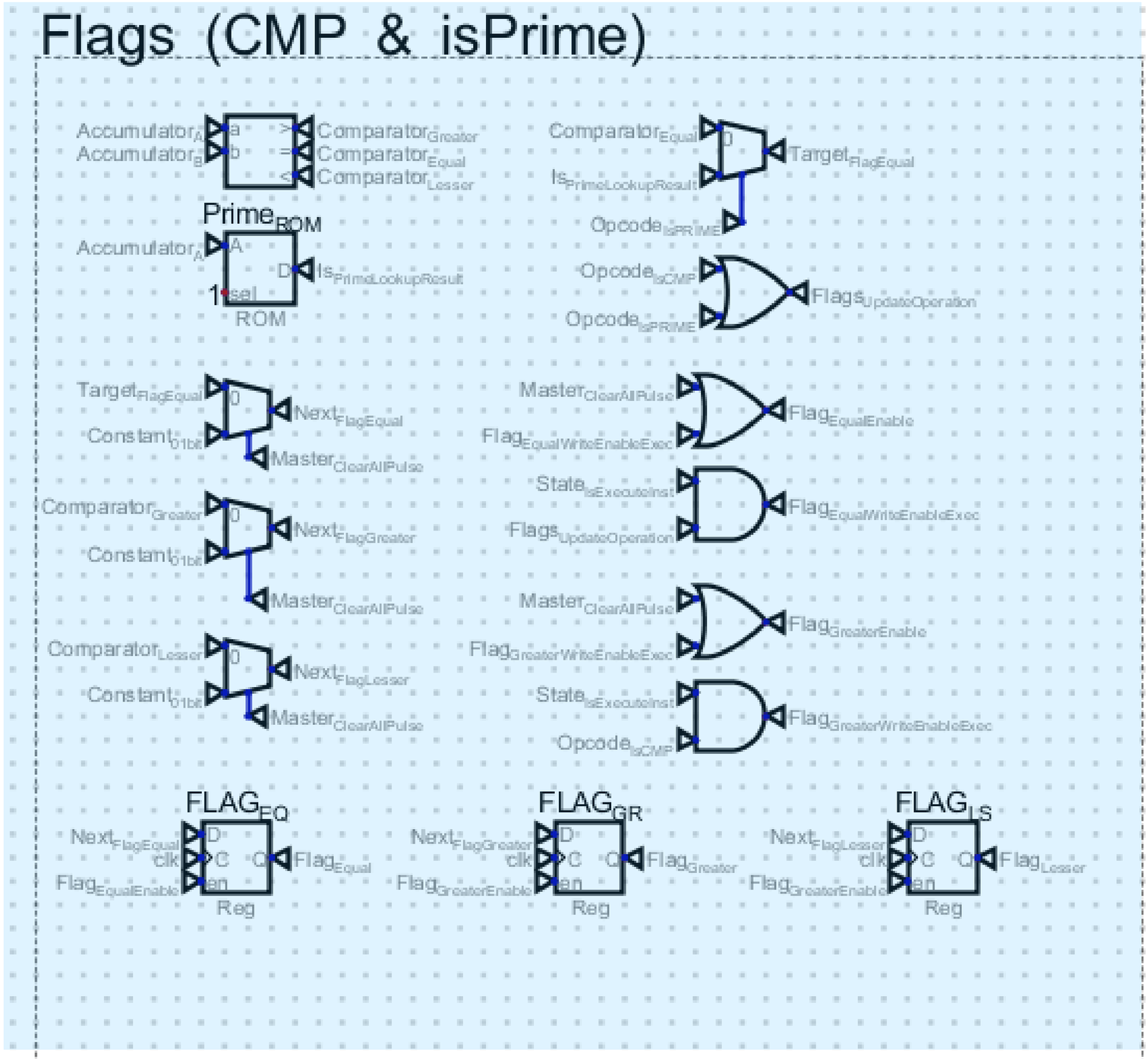


Stack, Registers & RAM Mux

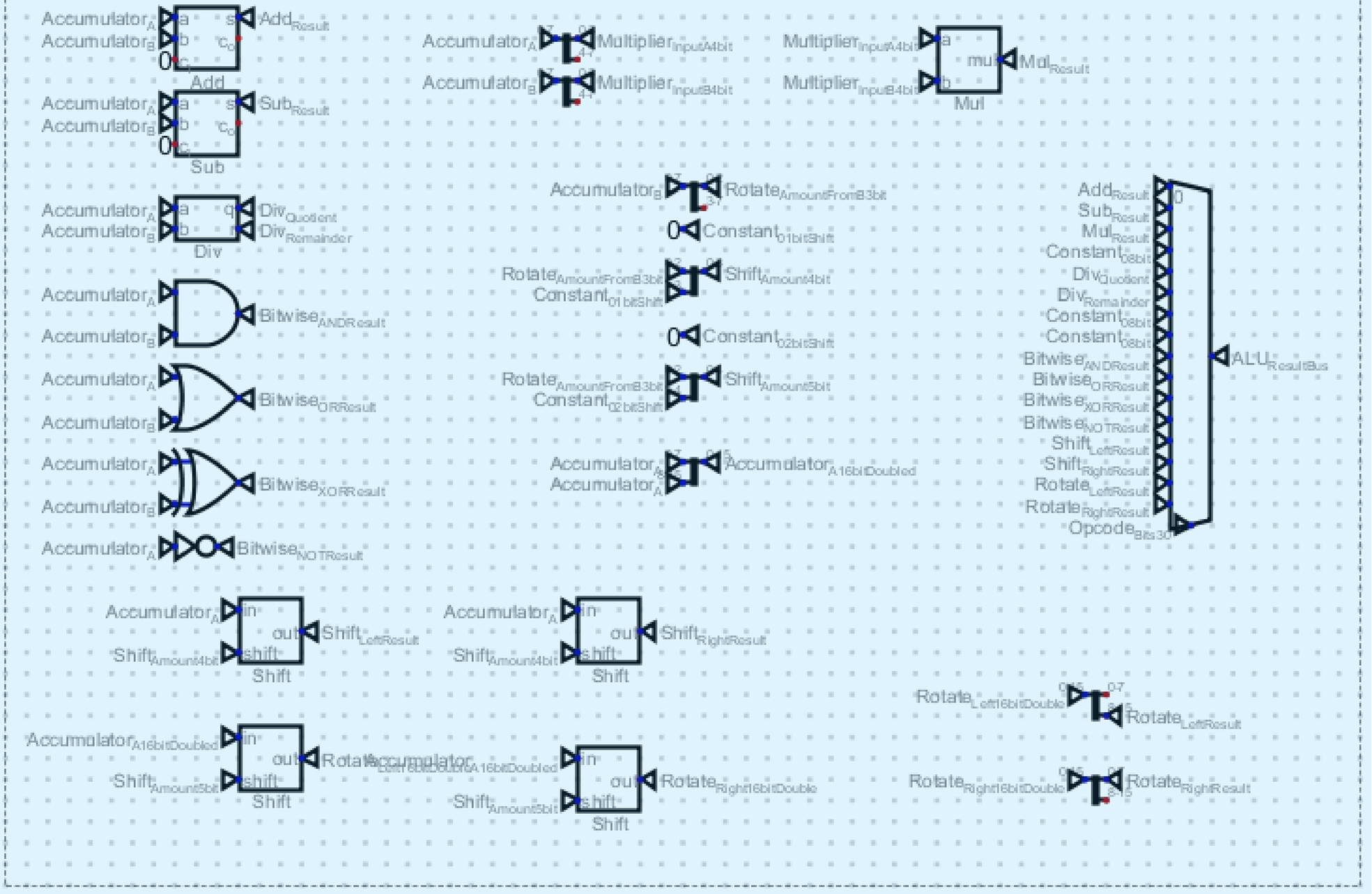


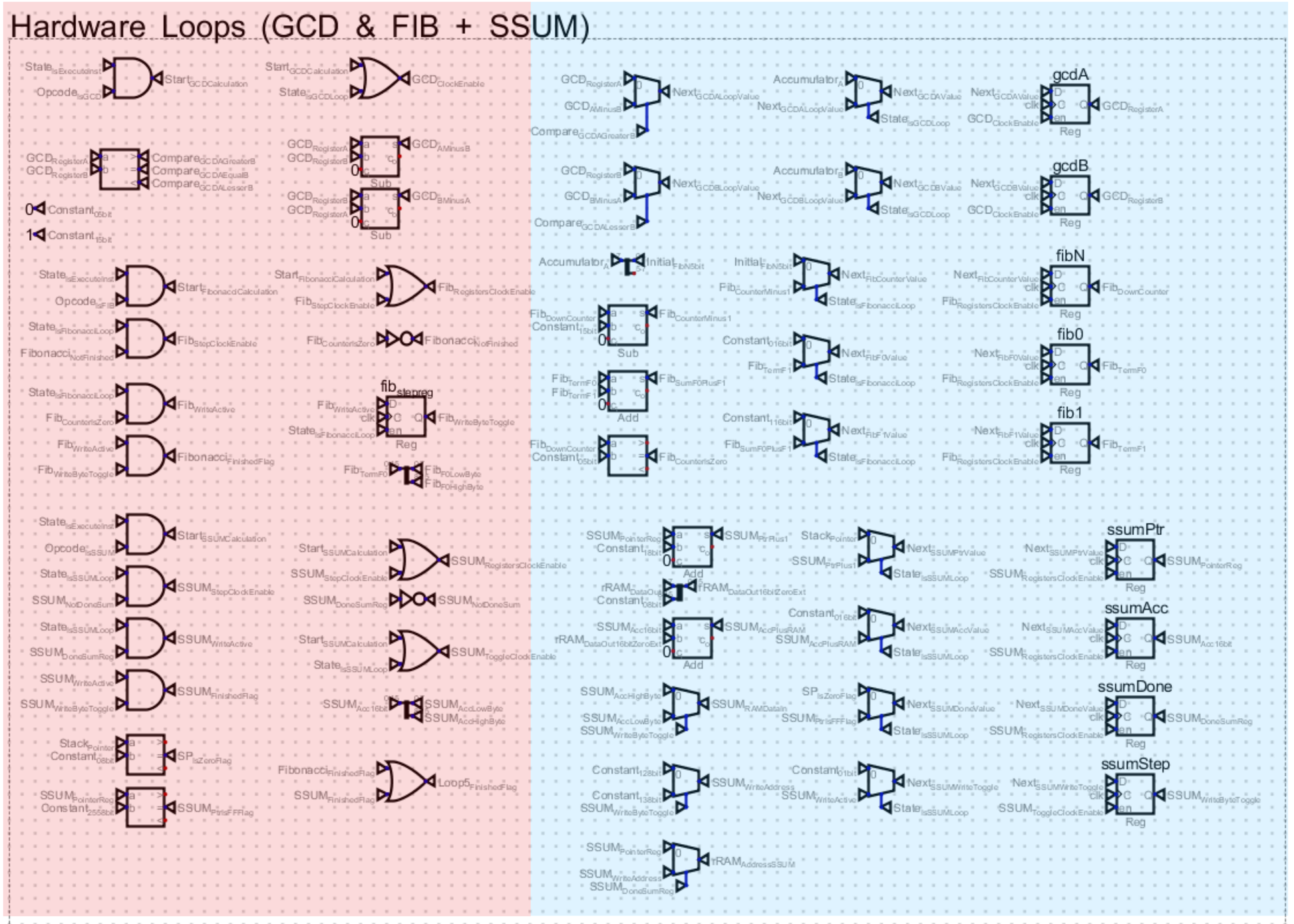
Branch & Program Counter (PC)




Flags (CMP & isPrime)
Accumulator_A
Accumulator_B
a
b
>
=
<
Comparator_Greater
Comparator_Equal
Comparator_Lesser
Prime_ROM
A
D
sel
ROM
Is_PrimeLookupResult
Comparator_Equal
Is_PrimeLookupResult
Target_FlagEqual
Opcode_isPRIME
Opcode_isCMP
Opcode_isPRIME
Flags_UpdateOperation
Target_FlagEqual
Constant_01b6
Next_FlagEqual
Master_ClearAllPulse
Comparator_Greater
Constant_01b6
Next_FlagGreater
Master_ClearAllPulse
Comparator_Lesser
Constant_01b6
Next_FlagLesser
Master_ClearAllPulse
Master_ClearAllPulse
Flag_EqualWriteEnableExec
Flag_EqualEnable
State_isExecuteInst
Flags_UpdateOperation
Flag_EqualWriteEnableExec
Master_ClearAllPulse
Flag_GreaterWriteEnableExec
Flag_GreaterEnable
State_isExecuteInst
Opcode_isCMP
Flag_GreaterWriteEnableExec
FLAG_EQ
Next_FlagEqual
clk
en
D
C
Q
Flag_Equal
Flag_EqualEnable
Reg
FLAG_GR
Next_FlagGreater
clk
en
D
C
Q
Flag_Greater
Flag_GreaterEnable
Reg
FLAG_LS
Next_FlagLesser
clk
en
D
C
Q
Flag_Lesser
Flag_GreaterEnable
Reg

ALU (Arithmetic Logic Unit)




Hardware Loops (GCD & FIB + SSUM)
gcdA
gcdB
fibN
fib0
fib1
ssumPtr
ssumAcc
ssumDone
ssumStep
Add
Sub
Reg



BYE BYE

จอมมารโบ๋ Will Return In Avengers: Doomsday



